# Lab S09. Que el aviso no tumbe la transferencia

Kevin Santiago González | Grupo 2
Sistemas Distribuidos 2027-1
23 de octubre de 2026

## 1. El problema

Primero probé la versión original. Apagué notificacion e hice una transferencia de 50 de Ana a Beto. El cliente recibió un 500, pero el dinero sí se movió. La transferencia 12 quedó en ABONADA.

| Cuenta | Antes | Después |
|---|---|---|
| Origen: 002180009277546301 | 750.00 | 700.00 |
| Destino: 002180009277546302 | 700.00 | 750.00 |

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % curl -i http://localhost:8080/api/cuentas/002180009277546301
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 5bbaa185404a
Content-Type: application/json
Date: Fri, 23 Oct 2026 17:18:54 GMT

{"id":6,"clabe":"002180009277546301","titular":"Ana","saldo":700.00}%
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % curl -i http://localhost:8080/api/cuentas/002180009277546302
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: aa53387cd005
Content-Type: application/json
Date: Fri, 23 Oct 2026 17:19:01 GMT

{"id":7,"clabe":"002180009277546302","titular":"Beto","saldo":750.00}%
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco %
```

Saldos después de hacer la transferencia con notificacion apagado.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180009277546301","claveDestino":"002180009277546302","monto":50}'
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Fri, 23 Oct 2026 17:14:42 GMT

{"timestamp":"2026-10-23T17:14:42.264+00:00","status":500,"error":"Internal Server Error","path":"/transferencias"}%
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco %
```

La petición respondió HTTP 500 aunque el abono ya estaba aplicado.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % curl -sS -w '\n' 'http://localhost:8080/api/transferencias?estado=ABONADA'
[{"id":12,"claveOrigen":"002180009277546301","claveDestino":"002180009277546302","monto":50.00,"fecha":"2026-10-23T17:14:41.795685Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":700.00,"intentosDeCompensacion":0,"actualizada":"2026-10-23T17:14:41.979522Z"}]
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco %
```

Estado guardado de la transferencia 12.

Pasó porque la transferencia todavía esperaba una respuesta HTTP de notificacion. El cargo y el abono ya estaban guardados cuando falló el aviso. El error no deshizo esos movimientos.

# 2. Publicar el evento

Quité la llamada HTTP a notificacion de TransferenciaService. Ahora guardo el estado final y después publico un evento en RabbitMQ. Declaré el exchange transferencias como topic y durable, y configuré RabbitTemplate con Jackson2JsonMessageConverter.

```
t.marcarCompletada();
t.prepararEvento(t.getSaldoOrigenTrasCargo(), destino.saldo());
t = guardar(t);
eventos.publicar(t);
```

Agregué eventoId a la transferencia. Se genera una sola vez y se guarda junto con los saldos del evento. Si lo vuelvo a publicar, conserva el mismo identificador. El mensaje se envía como persistente.

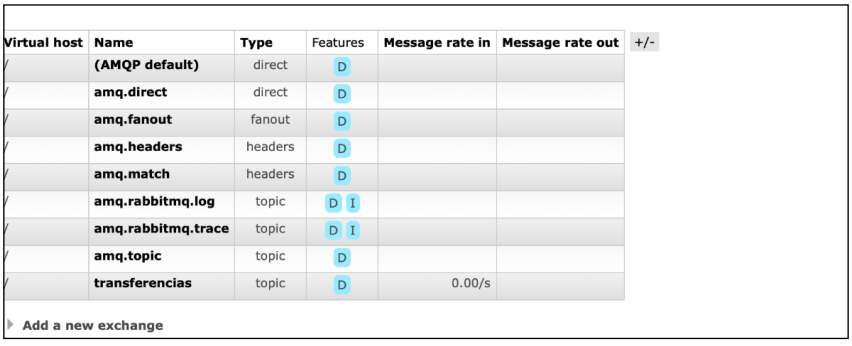
| Estado final | Evento / clave |
|---|---|
| COMPLETADA | TransferenciaRealizada / transferencia.realizada |
| COMPENSADA | TransferenciaRevertida / transferencia.revertida |

El JSON lleva eventoId, tipo, transferenciaId, clabeOrigen, clabeDestino, monto, saldoOrigen, saldoDestino y fecha. La petición HTTP sigue usando claveOrigen y claveDestino.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % docker compose ps -a notificacion
NAME                          IMAGE                          COMMAND              SERVICE        CREATED       STATUS                     PORTS
mexi-banco-v06-notificacion-1   mexi-banco-v06-notificacion   "java -jar app.jar"   notificacion   2 hours ago   Exited (137) 7 seconds ago
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % curl -i -X POST http://localhost:8080/api/transferencias \
  -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"0021800092775463O1","claveDestino":"0021800092775463O2","monto":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Fri, 23 Oct 2026 17:30:19 GMT

{"id":13,"claveOrigen":"0021800092775463O1","claveDestino":"0021800092775463O2","monto":50,"fecha":"2026-10-23T17:30:19.876031844Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":650.00,"int
entosDeCompensacion":0,"actualizada":"2026-10-23T17:30:20.477130469Z","eventoId":"3907d057-04e9-4ba1-9de2-9685343e262c","saldoOrigenEvento":650.00,"saldoDestinoEvento":800.00}%
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco %
```

Transferencia 13 COMPLETADA y HTTP 201 con notificacion apagado.



| Virtual host | Name | Type | Features | Message rate in | Message rate out | +/- |
|---|---|---|---|---|---|---|
| / | (AMQP default) | direct | D | | | |
| / | amq.direct | direct | D | | | |
| / | amq.fanout | fanout | D | | | |
| / | amq.headers | headers | D | | | |
| / | amq.match | headers | D | | | |
| / | amq.rabbitmq.log | topic | D I | | | |
| / | amq.rabbitmq.trace | topic | D I | | | |
| / | amq.topic | topic | D | | | |
| / | transferencias | topic | D | 0.00/s | | |

Add a new exchange

El exchange transferencias quedó como topic y durable.

```
transferencia-1  | 2026-10-23T17:30:20.508Z  WARN 1 --- [transferencia] [nectionFactory2] mx.mexibanco.mensajeria.RabbitConfig     : EVENTO SIN COLA eventoId=3907d057-04e9-4ba1-9de2-9685343e262c exchang
e=transferencias routingKey=transferencia.realizada motivo=NO_ROUTE
transferencia-1  | 2026-10-23T17:30:20.510Z  INFO 1 --- [transferencia] [nectionFactory2] mx.mexibanco.mensajeria.RabbitConfig     : PUBLISHER CONFIRM eventoId=3907d057-04e9-4ba1-9de2-9685343e262c ack=t
rue (no confirma consumo)
```

El mismo evento recibió NO_ROUTE y publisher confirm ack=true.

Activé publisher confirms. El ack confirma que el broker aceptó la publicación, no que notificacion haya guardado el aviso. En esta prueba todavía no había una cola enlazada: el mensaje no se conservó, aunque recibí la confirmación.

# 3. Consumir el evento

Declaré la cola durable notificacion.transferencias y la enlacé con transferencia.*. Arranqué notificacion para crearla y después lo apagué. Hice tres transferencias de 10, 20 y 30; las tres respondieron 201.

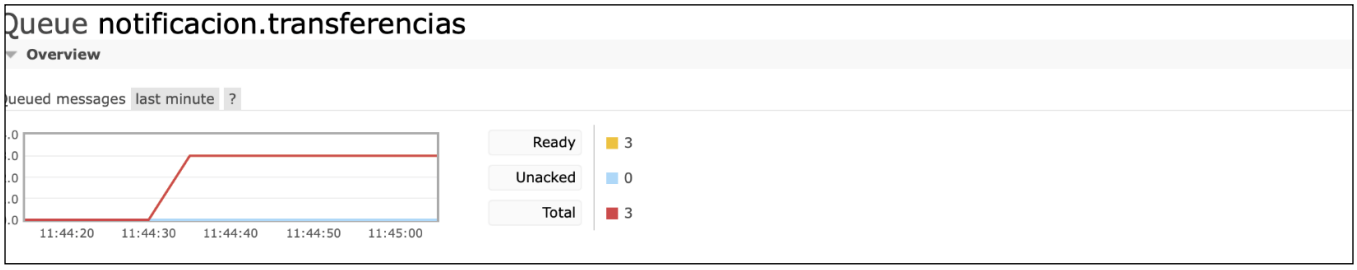
```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % for monto in 10 20 30; do
  curl --fail-with-body -i -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"002180009277546301\",\"claveDestino\":\"002180009277546302\",\"monto\":$monto}" || break
  printf '\n'
done
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Fri, 23 Oct 2026 17:44:29 GMT

{"id":14,"claveOrigen":"002180009277546301","claveDestino":"002180009277546302","monto":10,"fecha":"2026-10-23T17:44:29.175937875Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":640.00,"int
entosDeCompensacion":0,"actualizada":"2026-10-23T17:44:29.571761292Z","eventoId":"db469f57-7010-4979-b01e-40b07e2116e1","saldoOrigenEvento":640.00,"saldoDestinoEvento":810.00}
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Fri, 23 Oct 2026 17:44:29 GMT

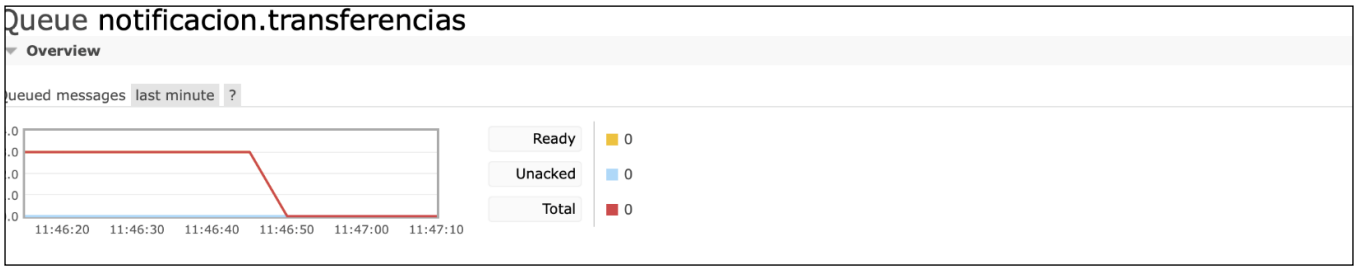
{"id":15,"claveOrigen":"002180009277546301","claveDestino":"002180009277546302","monto":20,"fecha":"2026-10-23T17:44:29.669600709Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":620.00,"int
entosDeCompensacion":0,"actualizada":"2026-10-23T17:44:29.774387542Z","eventoId":"6114825c-ddd5-4c0b-9456-4189c59b14e7","saldoOrigenEvento":620.00,"saldoDestinoEvento":830.00}
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Fri, 23 Oct 2026 17:44:29 GMT

{"id":16,"claveOrigen":"002180009277546301","claveDestino":"002180009277546302","monto":30,"fecha":"2026-10-23T17:44:29.828937001Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":590.00,"int
entosDeCompensacion":0,"actualizada":"2026-10-23T17:44:29.904779459Z","eventoId":"b1a27b73-7ba0-4177-b2cc-075f4583364d","saldoOrigenEvento":590.00,"saldoDestinoEvento":860.00}
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco %
```

Transferencias 14, 15 y 16 COMPLETADA mientras el consumidor estaba apagado.




La cola guardó los tres mensajes en Ready.




Al encender notificacion, la cola bajó de 3 mensajes a 0.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % curl -sS -w '\n' 'http://localhost:8080/api/notificaciones?clabe=002180009277546301'
[{"id":13,"clabeCuenta":"002180009277546301","mensaje":"Transferencia #16 enviada por 30 a 002180009277546302. Saldo: 590.00","fecha":"2026-10-23T17:46:43.441357Z"},{"id":11,"clabeCuenta":"0021800092775
46301","mensaje":"Transferencia #15 enviada por 20 a 002180009277546302. Saldo: 620.00","fecha":"2026-10-23T17:46:43.436703Z"},{"id":9,"clabeCuenta":"002180009277546301","mensaje":"Transferencia #14 env
iada por 10 a 002180009277546302. Saldo: 640.00","fecha":"2026-10-23T17:46:43.359001Z"},{"id":8,"clabeCuenta":"002180009277546301","mensaje":"SPEI_ENVIADO por 50, saldo resultante 750.00","fecha":"2026-
10-23T17:11:18.240134Z"},{"id":6,"clabeCuenta":"002180009277546301","mensaje":"TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00","fecha":"2026-10-23T17:11:11.643329Z"}]
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco % curl -sS -w '\n' 'http://localhost:8080/api/notificaciones?clabe=002180009277546302'
[{"id":14,"clabeCuenta":"002180009277546302","mensaje":"Transferencia #16 recibida por 30 de 002180009277546301. Saldo: 860.00","fecha":"2026-10-23T17:46:43.442718Z"},{"id":12,"clabeCuenta":"00218000927
7546302","mensaje":"Transferencia #15 recibida por 20 de 002180009277546301. Saldo: 830.00","fecha":"2026-10-23T17:46:43.438156Z"},{"id":10,"clabeCuenta":"002180009277546302","mensaje":"Transferencia #1
4 recibida por 10 de 002180009277546301. Saldo: 810.00","fecha":"2026-10-23T17:46:43.420404Z"},{"id":7,"clabeCuenta":"002180009277546302","mensaje":"TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.
00","fecha":"2026-10-23T17:11:12.524885Z"}]
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco %
```

Las dos bandejas recibieron los avisos de las tres transferencias.

```
notificacion-1  | 2026-10-23T17:46:43.433Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : AVISOS GUARDADOS Y ACK eventoId=db469f57-7010-4979-b01e-40b07e2116e1 tr
ansferenciaId=14 tipo=TransferenciaRealizada redelivered=false
notificacion-1  | 2026-10-23T17:46:43.439Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : AVISOS GUARDADOS Y ACK eventoId=6114825c-ddd5-4c0b-9456-4189c59b14e7 tr
ansferenciaId=15 tipo=TransferenciaRealizada redelivered=false
notificacion-1  | 2026-10-23T17:46:43.445Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : AVISOS GUARDADOS Y ACK eventoId=b1a27b73-7ba0-4177-b2cc-075f4583364d tr
ansferenciaId=16 tipo=TransferenciaRealizada redelivered=false
```

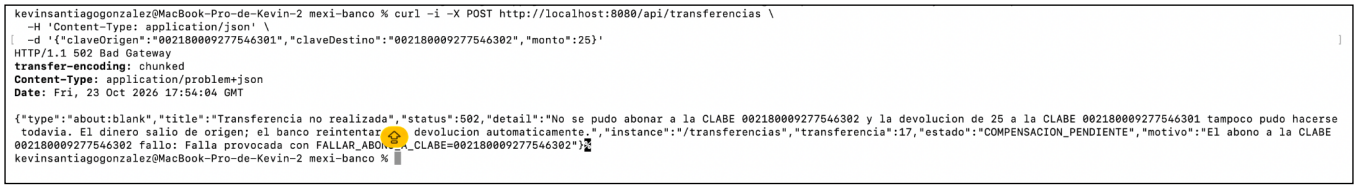
Logs del guardado de avisos y del ACK de cada evento.

El listener usa ACK manual. Llama a un procesador con @Transactional, que guarda los dos avisos de una transferencia realizada en la misma transacción. El ACK se manda después de que esa transacción termina. Así, si falla el guardado, el evento puede volver a entregarse.

La cola permitió que los eventos esperaran. Transferencia y notificacion ya no tuvieron que estar encendidos al mismo tiempo.

# 4. El aviso de devolución

Puse FALLAR_ABONO_A_CLABE con la CLABE de Beto e hice una transferencia de 25. El abono falló como esperaba, pero la transferencia 17 quedó primero en COMPENSACION_PENDIENTE.

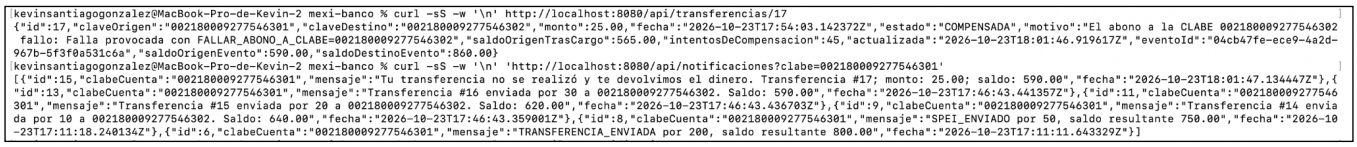



Respuesta inicial de la transferencia 17: HTTP 502 y compensación pendiente.

Revisé el saldo y los logs. Ana ya tenía otra vez 590, pero movimiento respondía 500 al guardar DEVOLUCION_TRANSFERENCIA. La restricción movimiento_tipo_check de PostgreSQL no aceptaba ese tipo.

```
BEGIN;
ALTER TABLE public.movimiento
  DROP CONSTRAINT movimiento_tipo_check;
ALTER TABLE public.movimiento
  ADD CONSTRAINT movimiento_tipo_check CHECK (tipo IN (
    'DEPOSITO', 'TRANSFERENCIA_ENVIADA',
    'TRANSFERENCIA_RECIBIDA', 'SPEI_ENVIADO',
    'DEVOLUCION_TRANSFERENCIA'
  ));
COMMIT;
```

Actualicé la restricción sin borrar los movimientos. No hice otra transferencia ni devolví dinero manualmente. El proceso de reintentos de la base S09 terminó la misma operación y guardó COMPENSADA.



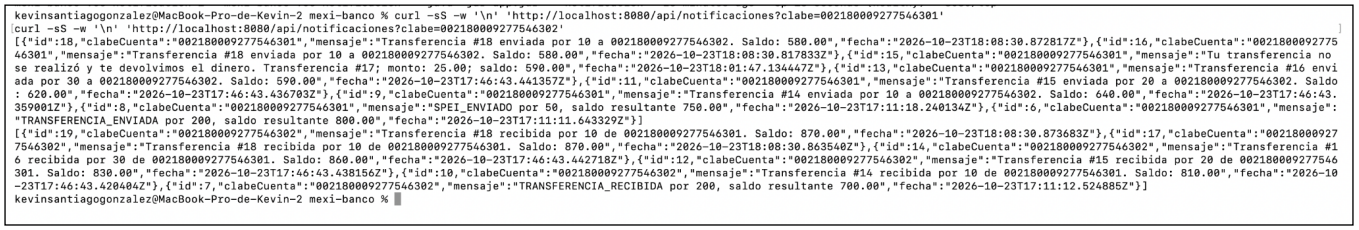

Transferencia 17 COMPENSADA y aviso de devolución en la bandeja de Ana.

| Cuenta | Antes de la prueba | Después |
|---|---|---|
| Ana | 590.00 | 590.00 |
| Beto | 860.00 | 860.00 |

Llegó un solo aviso de TransferenciaRevertida al origen, con el mensaje de que la transferencia no se realizó y el dinero fue devuelto. Después quité la falla de prueba.

# 5. El duplicado y mi arreglo

Con notificacion apagado, copié el evento de la transferencia 18 desde Get messages usando Nack message requeue true. Después publiqué el mismo JSON en transferencias, con la misma clave y el mismo eventoId.




Antes del arreglo: dos avisos de la transferencia 18 en cada bandeja.

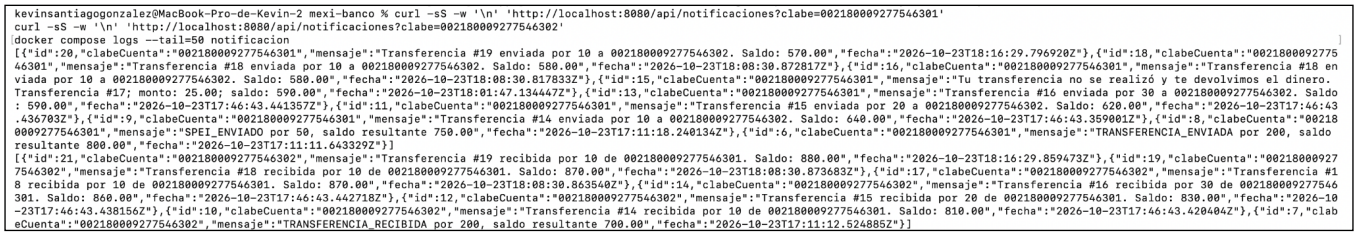
| Cuenta | Primer aviso | Aviso repetido |
|---|---|---|
| Ana | 16 | 18 |
| Beto | 17 | 19 |

Al encender el consumidor, guardó los avisos dos veces. No se repitió el movimiento de dinero; se repitió el procesamiento del evento. Para corregirlo agregué la tabla evento_procesado, con evento_id como clave primaria.

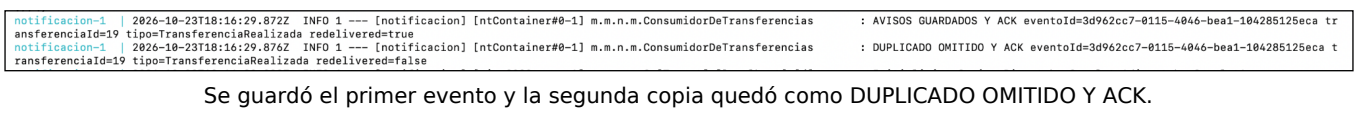
```
INSERT INTO evento_procesado (evento_id, procesado_en)
VALUES (:eventoId, CURRENT_TIMESTAMP)
ON CONFLICT (evento_id) DO NOTHING;

// Dentro del procesador con @Transactional:
if (procesados.registrarSiNuevo(e.eventoId()) == 0) {
    return false;
}
// Después se guardan los avisos en la misma transacción.
```

Si el evento ya existe, no guardo más avisos y hago ACK. Si falla el guardado, también se revierte el registro del evento. La restricción de la base evita que dos consumidores registren la misma clave al mismo tiempo.




Después del arreglo: la transferencia 19 tiene un solo aviso por cuenta.




Se guardó el primer evento y la segunda copia quedó como DUPLICADO OMITIDO Y ACK.

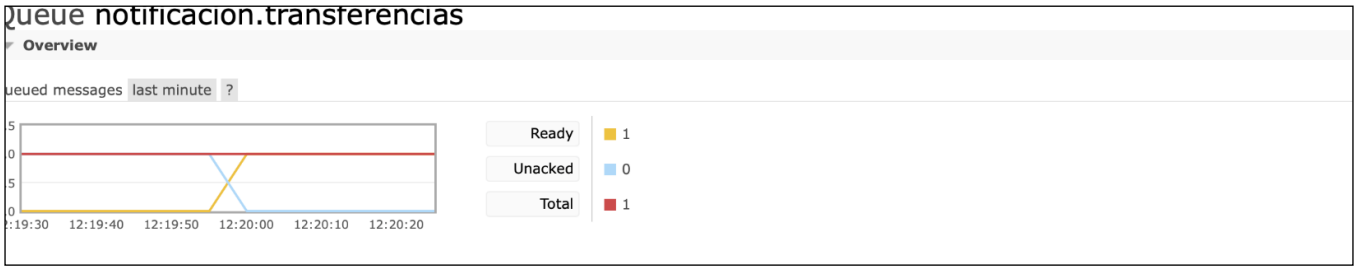
Repetí la prueba con un evento nuevo: el de la transferencia 19. Publiqué dos copias y solo se crearon los avisos 20 para Ana y 21 para Beto. La cola terminó en 0. Conservé los duplicados anteriores como evidencia.

# 6. El mensaje envenenado

Publiqué el texto incompleto {"hola": en transferencias con la clave transferencia.realizada. Antes del arreglo, el consumidor lo intentaba leer una y otra vez. Los logs mostraban JsonEOFException, redelivered=true y requeue=true.




El mensaje inválido se reencolaba y repetía el mismo error.



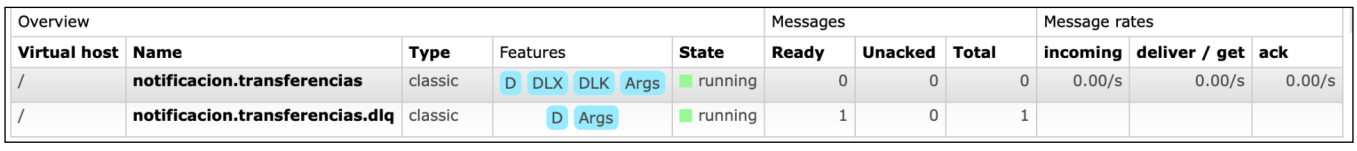

Al apagar notificacion quedó un mensaje Ready.

Agregué un exchange para mensajes rechazados y una DLQ. Borré la cola anterior, que solo tenía el mensaje inválido de prueba, para poder crearla con los argumentos nuevos. Después volví a publicar el mismo texto.

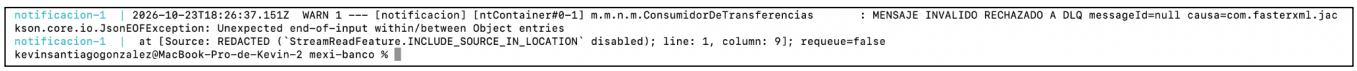
| Pieza | Configuración |
|---|---|
| DLX | transferencias.dlx, direct y durable |
| DLQ | notificacion.transferencias.dlq, durable |
| Clave hacia la DLQ | notificacion.transferencias |

La cola principal usa x-dead-letter-exchange=transferencias.dlx y x-dead-letter-routing-key=notificacion.transferencias. Si el JSON no se puede leer o no cumple el contrato, lo rechazo sin reencolar:

```
canal.basicNack(tag, false, false);
```




Después del arreglo: la principal quedó en 0 y la DLQ recibió 1 mensaje.

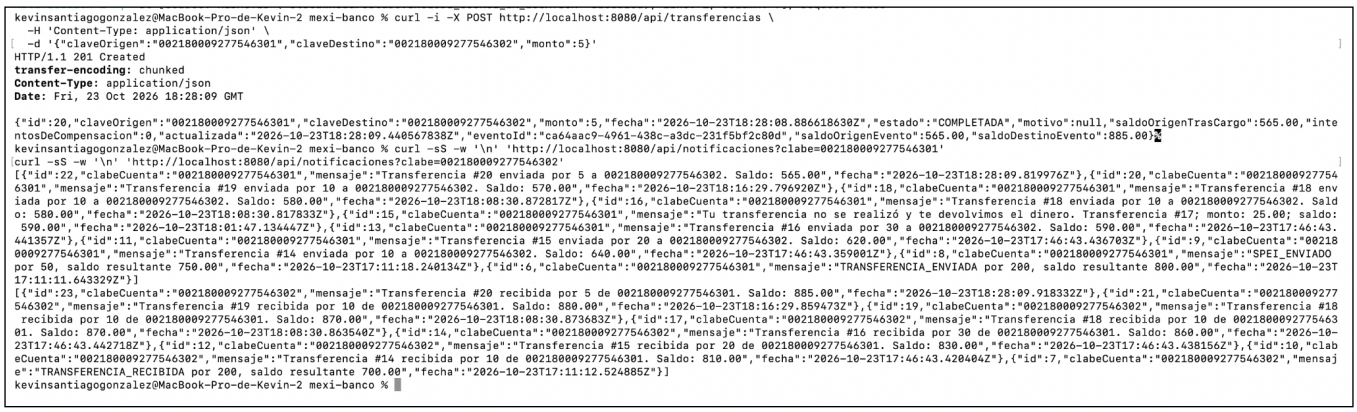



El rechazo quedó registrado con requeue=false.

Separé ese caso de los errores al guardar. Un fallo de la base sigue reintentándose; un JSON inválido se aparta para revisarlo y deja de regresar a la cola principal.

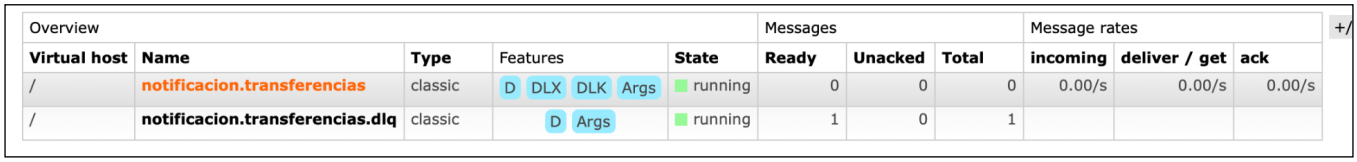
# 7. La transferencia normal sigue funcionando

Después de mandar el mensaje inválido a la DLQ hice una transferencia normal de 5. La transferencia 20 respondió 201 y quedó COMPLETADA. Ana recibió el aviso 22 y Beto el aviso 23.




Respuesta de la transferencia 20 y sus avisos en ambas bandejas.

| Cuenta | Antes | Después |
|---|---|---|
| Ana | 570.00 | 565.00 |
| Beto | 880.00 | 885.00 |



| Overview | | | | | Messages | | | Message rates | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Virtual host | Name | Type | Features | State | Ready | Unacked | Total | incoming | deliver / get | ack |
| / | notificacion.transferencias | classic | D DLX DLK Args | running | 0 | 0 | 0 | 0.00/s | 0.00/s | 0.00/s |
| / | notificacion.transferencias.dlq | classic | D Args | running | 1 | 0 | 1 | | | |

La cola principal siguió vacía y el mensaje inválido permaneció en la DLQ.

El consumidor pudo seguir con un evento válido. El mensaje malo quedó separado y no impidió que se guardaran los avisos de la nueva transferencia.

### Lo que comprobé

| Prueba | Resultado |
|---|---|
| Notificación apagado | La transferencia terminó y los eventos esperaron en la cola. |
| Mismo evento dos veces | Después del arreglo quedó un solo juego de avisos. |
| JSON incompleto | Se rechazó sin reencolar y llegó a la DLQ. |
| Evento válido después del error | Se guardó un aviso por cuenta y la principal quedó vacía. |

Hice las pruebas manuales mostradas. La construcción Docker omitió las pruebas automáticas; no presento la suite como ejecutada. No implementé el reto opcional de Outbox.

# 8. Preguntas de cierre

### Si RabbitMQ está caído

En mi versión guardo COMPLETADA antes de publicar. Si RabbitMQ está caído, ese estado puede quedar guardado y la petición puede responder con error al intentar enviar el evento. El aviso no se reenvía solo después. El publisher confirm no une la escritura de la base con la publicación y tampoco confirma que notificacion haya guardado el aviso. En la primera prueba recibí ack=true junto con NO_ROUTE porque todavía no había cola. Para resolver la doble escritura tendría que guardar un Outbox en la misma transacción y publicar sus eventos pendientes después. Esto no está implementado.

### Por qué hago ACK después de guardar

Si hago ACK primero y el proceso se cae antes de guardar, el broker puede retirar el mensaje aunque el aviso no exista. Si guardo primero y hago ACK después, una caída entre esos pasos puede provocar una entrega repetida. Por eso guardo el eventoId con los avisos en la misma transacción. Cuando llega otra copia, la reconozco y la confirmo sin escribir avisos nuevos. No evito que el mensaje se entregue varias veces; evito repetir su efecto en la bandeja.

### Lo que elegí y lo que sacrifiqué

Elección: cambié la llamada HTTP a notificacion por un evento en RabbitMQ.
Porque: un aviso pendiente no debe provocar el error de una transferencia que ya movió dinero.
Sacrifico: la confirmación inmediata del aviso; puede llegar después y tengo que manejar duplicados y mensajes inválidos.

### Lo que todavía falta

La versión no cubre todos los fallos posibles. Los errores al guardar se reintentan sin un límite y la publicación no tiene recuperación automática si falla el broker. La DLQ aparta el mensaje inválido en este montaje, pero no garantiza que nunca se pierda un mensaje ante cualquier caída.

### Referencias

Consigna del Lab S09, Sistemas Distribuidos 2027-1, grupo 2.
RabbitMQ: rabbitmq.com/docs/confirms y rabbitmq.com/docs/dlx.
PostgreSQL: postgresql.org/docs/17/sql-insert.html.
Spring Data JPA: docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html.